Context-Aware Multi-Task V2X Message Classification Using CNN-Based Recurrent Architectures

Sumit Bhati
Department of Computer Science and Engineering
Bennett University, Greater Noida, India

***Abstract*—Vehicle-to-Everything (V2X) communication supports the exchange of safety, traffic, road-condition, and emergency information among vehicles and other transportation entities. Automated interpretation of these messages can assist downstream monitoring and prioritisation. This paper documents a comparative prototype for V2X message analysis using four convolutional-recurrent neural network variants: CNN-BiGRU, CNN-BiLSTM, CNN-StackLSTM, and CNN-BiRNN. The common model encodes message text using a vocabulary-limited vectorisation layer, an embedding layer, one-dimensional convolution, and max pooling. A separate branch embeds message priority; the two representations are concatenated and passed to prediction heads for six-way message-category classification and binary spam prediction. The current development experiments use a 12,000-row synthetic, realistic-inspired dataset with 2,000 examples per category. Earlier project logs report perfect scores for all four models, but these values have not been independently reproduced. More importantly, the spam indicator is deterministically aligned with the message category, creating a target shortcut, while the synthetic data limits external validity. The manuscript therefore presents the implementation and its preliminary status rather than claiming validated real-world performance. It identifies the data and experimental checks required before drawing comparative conclusions.**

***Index Terms*—V2X, text classification, convolutional neural networks, GRU, LSTM, multi-task learning, message priority.**

I. Introduction

Vehicle-to-Everything (V2X) communication enables information exchange among vehicles, roadside infrastructure, pedestrians, and network services. Potential message types include collision or emergency warnings, roadwork notifications, weather alerts, hazard reports, and traffic advisories. Automated classification of such messages is relevant to intelligent transportation systems because the content and urgency of received messages can influence subsequent processing.

Lazaga *et al.* studied V2X message categorisation using recurrent and convolutional-recurrent networks and reported results for BRNN, CNN-BRNN, BLSTM, and CNN-BLSTM models [1]. The implementation documented here takes a different direction: it compares four CNN-based recurrent variants, introduces priority as a second model input, and provides two prediction outputs. The task is described as *multi-task classification*: one output is a six-class categorical prediction and the other is a binary prediction. It is not a six-label multi-label classification problem because the category head uses a Softmax output.

The objectives are to: (1) implement a common pipeline for six V2X message categories; (2) compare four CNN-recurrent variants under shared preprocessing; (3) encode message priority as contextual input; and (4) document the limitations that must be addressed before reporting a defensible empirical comparison.

A. Operational Context and Intended Role

A V2X message may communicate an event whose relevance depends on both its wording and its stated priority. For example, a short collision-warning message and a routine traffic advisory may contain overlapping road-related vocabulary but require different levels of attention. In this prototype, message content provides the semantic signal, while the priority field supplies an explicit contextual feature. The model is intended to support message organisation and review; it is not a vehicle-control system, a safety-certified warning engine, or a replacement for communication standards and operational validation.

The implemented demonstration accepts a message and its priority, then returns a predicted category and a model-derived spam score. This is an application-level analysis workflow. The current code does not establish a live connection to an onboard unit, roadside unit, or production V2X network. Figure 1 places the prototype in this limited role and distinguishes the incoming information from the model's outputs.

II. Related Work and Research Context

The reference study considers six categories: Spam, Emergency, Roadwork, Weather, Hazard, and Traffic. It describes tokenisation, padding, embedding, recurrent models, and convolutional-recurrent hybrids. Its reported accuracy values are 88.98% for BRNN, 90.00% for CNN-BRNN, 74.44% for BLSTM, and 73.92% for CNN-BLSTM [1]. These are results from that paper's dataset and protocol; they are not results of the present implementation and should not be compared as if both studies used the same experimental conditions.

Surveys of text classification discuss convolutional, recurrent, and hybrid approaches for learning representations from sequences [3], [4]. Convolutional layers can capture local token patterns, while recurrent units model sequential dependen-

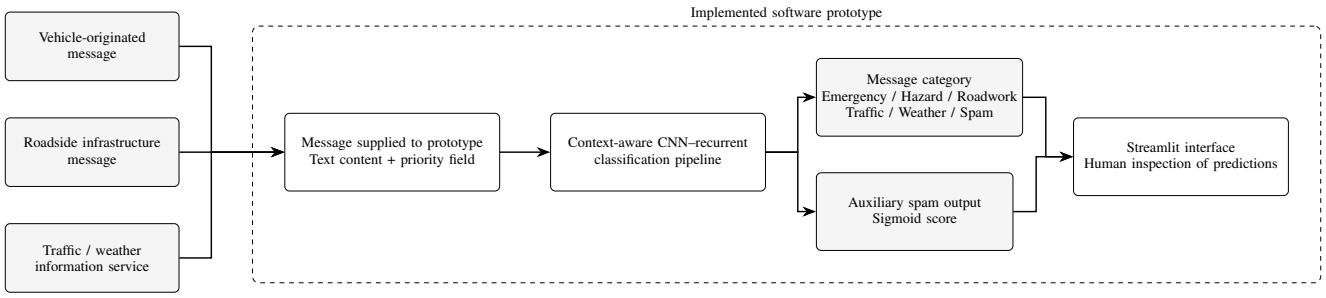


Fig. 1. Conceptual V2X information context and the implemented prototype boundary. Vehicle, roadside, and service sources illustrate possible message origins; the current implementation receives supplied message text and priority and does not connect directly to live V2X equipment.

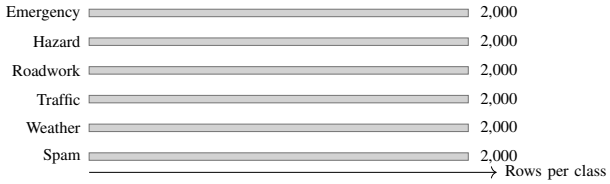


Fig. 2. Reported category counts in the 12,000-row synthetic development dataset.

cies. Gated recurrent units and long short-term memory networks are commonly used recurrent alternatives for sequence modelling [5]. These properties motivate comparing recurrent blocks while keeping the remaining model components fixed.

III. Dataset and Problem Formulation

A. Development Dataset

The development file is named `V2X_realistic_inspired_dataset_12000.csv`. Project records describe 12,000 rows, 12,000 unique message contents, no exact duplicate rows, and a balanced distribution of 2,000 examples in each of six message categories. The dataset is synthetic and realistic-inspired; it is not a collection of raw messages captured from operational vehicles or roadside units. This distinction limits the conclusions that can be drawn from its results.

The model uses message content and priority as inputs. Message Type is the six-class target, with categories Emergency, Hazard, Roadwork, Traffic, Weather, and Spam. A separate Spam field is used as a binary target.

Figure 2 summarises the reported class balance. The equal counts describe the synthetic development file only; they should not be interpreted as the natural frequency of events in deployed V2X traffic.

B. Label Structure and Shortcut Risk

In the current dataset, the Spam field is deterministically aligned with Message Type: examples in the Spam category are labelled positive, while examples in the other five categories are labelled negative. Consequently, the binary target restates information already present in the six-class target. A high spam score may therefore result from learning the message category rather than an independent spam-detection capability. The auxiliary output should not be presented as an independent detection contribution unless the label structure is revised or evaluated on data where spam status is not a deterministic function of message category.

C. Preprocessing and Partitioning

The implementation checks required columns, removes rows with missing required values, removes exact duplicate records, trims surrounding whitespace, removes empty messages, and retains the six configured categories. Category identifiers are assigned as Emergency=0, Hazard=1, Roadwork=2, Traffic=3, Weather=4, and Spam=5. Binary spam labels are normalised from common Yes/No, True/False, and 1/0 representations. Priorities outside Low, Medium, and High are mapped to Unknown.

The split is performed at the unique-message level. A 70/30 split is followed by an equal division of the remaining 30%, giving nominal training, validation, and test proportions of 70/15/15. The split uses a fixed random state of 42 and stratification by message category. Keeping exact message contents in a single partition does not rule out near-duplicate or shared-template leakage.

IV. Proposed Architecture

A. Text and Priority Encoders

The text branch uses a Keras `TextVectorization` layer with a maximum vocabulary size of 2,000 and a fixed sequence length of 40. The vectoriser is adapted using training messages. Token identifiers are mapped to trainable embeddings of dimension 64. A one-dimensional convolution with 64 filters, kernel size 3, same padding, and ReLU activation is followed by max pooling with pool size 2.

The priority branch maps priority values through a `StringLookup` layer and an 8-dimensional trainable embedding. The complete dual-input architecture is shown in Fig. 4; the feature-fusion operation is isolated in Fig. 3. The flattened priority representation is concatenated with the recurrent text representation. A dense layer with 64 ReLU units and dropout of 0.25 processes the fused representation.

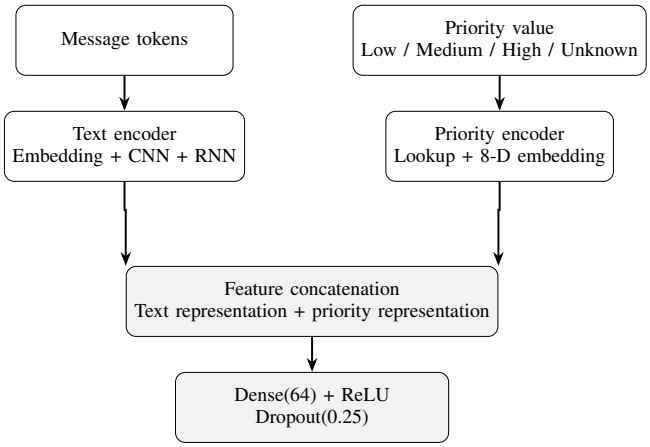


Fig. 3. Priority-aware feature fusion. Priority is encoded separately and concatenated with the learned text representation before the shared prediction layers.

TABLE I
RECURRENT COMPONENTS COMPARED IN THE IMPLEMENTATION

Variant	Recurrent component
CNN-BiGRU	Bidirectional GRU, 64 units per direction
CNN-BiLSTM	Bidirectional LSTM, 64 units per direction
CNN-StackLSTM	LSTM(64, sequences) followed by dropout (0.25) and LSTM(32)
CNN-BiRNN	Bidirectional SimpleRNN, 64 units per direction

B. Recurrent Variants

All four models share the same text front-end, priority branch, fusion layer, and prediction heads. They differ in the recurrent block, as summarised in Table I.

C. Prediction Heads and Objective

The fused representation feeds two outputs. The category head has six Softmax units and uses sparse categorical cross-entropy. The binary head has one Sigmoid unit and uses binary cross-entropy. Adam is the optimiser. The current compile configuration does not specify task-specific loss weights; the total loss is therefore described as the sum of the two configured losses:

$$\mathcal{L} = \mathcal{L}_{\text{type}} + \mathcal{L}_{\text{spam}}. \tag{1}$$

This implementation detail does not remove the target redundancy described in Section III-B.

D. End-to-End Experimental Workflow

The project code separates data preparation, model construction, training, evaluation, prediction, and interactive demonstration. Figure 5 summarises the sequence. The split is created once from unique message contents, and the four architectures are trained and evaluated using the same partitioning procedure. The saved models can subsequently be loaded for single-message prediction.

TABLE II
PREVIOUSLY LOGGED DEVELOPMENT VALUES (UNVERIFIED)

Architecture	Reported scores*	Runtime (s)*
CNN-BiGRU	1.000	34.67
CNN-BiLSTM	1.000	34.65
CNN-StackLSTM	1.000	36.96
CNN-BiRNN	1.000	52.55

V. EXPERIMENTAL PROTOCOL

The training routine iterates over the four architecture variants. For each model, the vectoriser is adapted on the training partition, the model is trained using the training data, and the validation partition is supplied to the training procedure. The default batch size recorded in the project code is 32; the epoch count is configurable. The routine records elapsed training time, evaluates the held-out test partition, saves the Keras model, and writes model-specific and combined JSON result files.

The recorded evaluation includes category accuracy, macro precision, macro recall, macro F1, binary spam accuracy at a 0.5 threshold, training runtime, a category confusion matrix, and a category classification report. A stronger final evaluation should also report per-class precision, recall, and F1; a spam confusion matrix and spam precision/recall/F1; model parameter counts; inference latency; software and hardware versions; and variability over multiple random seeds. All architectures should use identical partitions and comparable training conditions.

VI. PRELIMINARY DEVELOPMENT RESULTS

Prior project logs record a run on the synthetic development dataset in which all four variants achieved 1.000 for category accuracy, precision, recall, F1, and spam accuracy. The logged runtimes are shown in Table II. These values have not been independently reproduced during preparation of this manuscript. They are retained only as preliminary development records and must not be interpreted as validated experimental findings.

*The score is the logged value for category accuracy, macro precision, macro recall, macro F1, and spam accuracy. Values are unverified and not independently reproduced.

The identical perfect scores across all variants warrant investigation rather than a claim of model superiority. Class-specific lexical patterns, synthetic template regularities, and the deterministic spam label may make the task easier than the intended deployment problem. The results must be rerun after a template-overlap audit and on an independently sourced dataset.

VII. DISCUSSION

The implementation provides a common framework in which the recurrent block changes while the remaining architecture is held constant. This supports a controlled comparison in principle. The priority branch explicitly supplies a contextual feature to the model, but the current results do not establish that priority improves classification. That claim

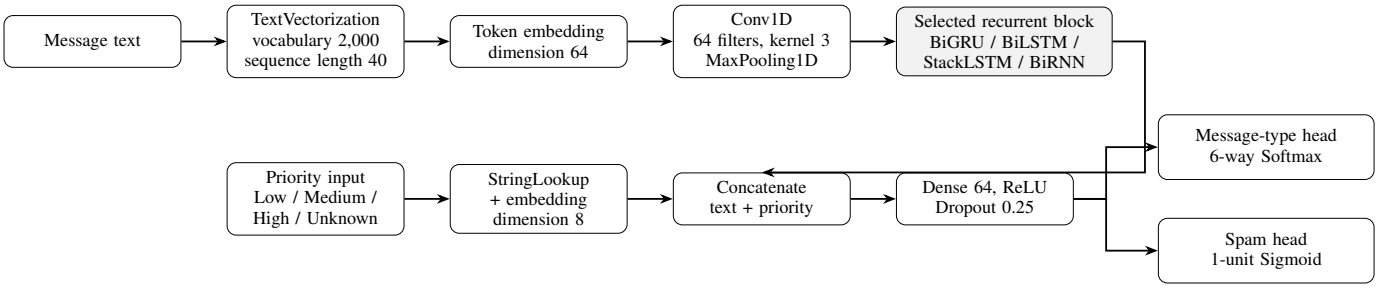


Fig. 4. Implemented dual-input, dual-output model. The recurrent block is varied across four experiments; the remaining components are shared.

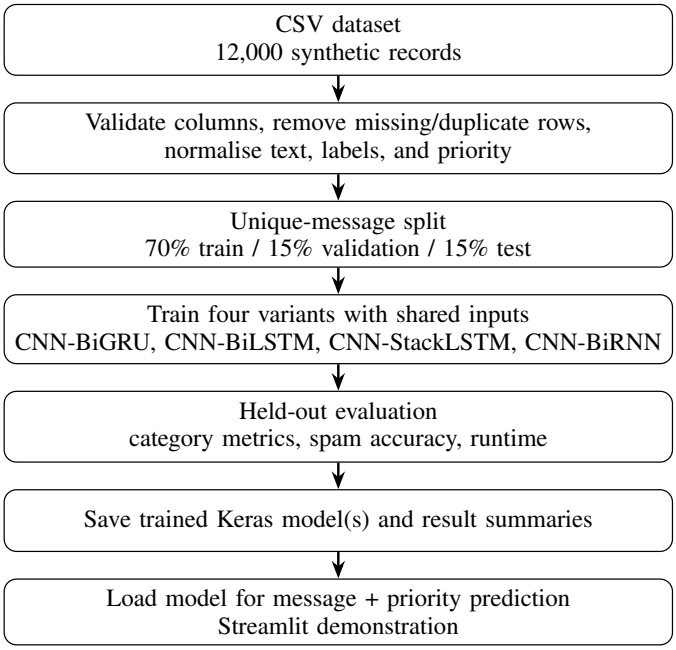


Fig. 5. Implementation workflow derived from the consolidated project code.

requires an ablation comparing otherwise identical models with and without the priority branch.

There is also a padding concern. The implementation uses an embedding configuration with `mask_zero=False`; the subsequent convolution does not automatically preserve sequence-padding masks. Padded positions may therefore influence convolutional features. A padding-aware alternative should be tested. In addition, the maximum Softmax score is a model output score and should not be described as a calibrated probability without a separate calibration evaluation.

VIII. Limitations and Future Work

The present study has several limitations. First, the development dataset is synthetic rather than operational V2X traffic. Second, the spam target duplicates the category label and therefore cannot establish independent spam detection. Third, exact duplicate removal does not rule out near-duplicate templates across partitions. Fourth, the logged perfect metrics and runtimes have not been independently reproduced. Fifth, padding behaviour, multiple-seed variability, inference latency, and parameter counts remain to be evaluated.

The intended IEEE DataPort dataset must be obtained under its access conditions and its schema and labels inspected before it can be described as an experimental dataset used in this work. Future experiments should audit template overlap, revise or remove the redundant spam target, compare models with and without priority, test padding-aware processing, repeat runs across multiple seeds, and report complete per-class and binary-task metrics.

IX. Conclusion

This paper documents a context-aware V2X message-classification prototype with a CNN-recurrent text encoder, a learned priority representation, and two prediction heads. Four recurrent variants are implemented in a common pipeline. The current development records show perfect scores on synthetic data, but the deterministic relationship between message category and spam label, the possibility of template shortcuts, and the lack of independent reproduction prevent those values from supporting real-world performance claims. Further dataset validation and controlled experiments are required before selecting a best-performing architecture or making deployment claims.

References

[1] C. F. Lazaga, M. A. Rosales, and G. P. Tan, "V2X Message Multi-label Classification: A Deep Learning Approach," in *2025 1st International Conference on Artificial Intelligence Technology (ICoAIT)*, 2025, pp. 137–142, doi: 10.1109/ICoAIT67446.2025.11308890.

[2] M. R. Jivthesh, N. B. Sai Shibu, and S. N. Rao, "V2X Message Classification, Prioritization, and Spam Detection Dataset," IEEE DataPort, 2023, doi: 10.21227/yhfd-0d92.

[3] S. Minaee *et al.*, "Deep Learning-based Text Classification: A Comprehensive Review," *ACM Computing Surveys*, vol. 54, no. 3, Art. no. 62, 2021, doi: 10.1145/3439726.

[4] A. Gasparetto, M. Marcuzzo, A. Zangari, and A. Albarelli, "A Survey on Text Classification Algorithms: From Text to Predictions," *Information*, vol. 13, no. 2, Art. no. 83, 2022, doi: 10.3390/info13020083.

[5] "Advancements and Challenges in Gated Recurrent Units (GRU) for Text Classification: A Systematic Literature Review," in *2024 7th International Conference of Computer and Informatics Engineering (IC2IE)*, 2024, doi: 10.1109/IC2IE63342.2024.10748229.